

Memoria Tecnica

EcoRecycle Tech SA — Planta de Clasificacion Automatizada

Alumno: Adrian Murciego Lobato | Asignatura: POO | Universidad: UTAMED | Curso: 2025-2026

1. Decisiones de diseno

1.1 Jerarquia de clases

Se ha disenado una jerarquia de tres niveles para modelar los residuos de la planta. En la cima se encuentra la interfaz **IResiduo**, que define el contrato minimo (getID, getPeso, getTipo) y permite gestionar colecciones homogeneas (List<IResiduo>) con polimorfismo puro. Debajo, la clase abstracta **Residuo** implementa IResiduo y centraliza los atributos comunes (id, peso, toxico), exponiendo el metodo abstracto **esReciclable()** que cada subclase concreta sobreescribe con su propia logica de negocio.

Las clases concretas son **ResiduoPlastico** (con subtipo PET/HDPE/PVC/LDPE), **ResiduoVidrio** (con color), **ResiduoPapel** (con indicador de humedad) y **ResiduoMetal** (ferroso/no ferroso). Cada una decide autonomamente si es reciclable: el plastico requiere no ser toxico y pesar menos de 10 kg; el vidrio solo exige no ser toxico; el papel anade la restriccion de no estar mojado; el metal es reciclable si no es toxico.

1.2 Extensibilidad (Principio Abierto/Cerrado)

Para demostrar el principio OCP de SOLID, se anadio **ResiduoMetal** y su correspondiente **Deposito de Metal** sin modificar ninguna clase existente. El unico punto de extension fue anadir una entrada al enum TipoResiduo y un case en ResiduoFactory, mas una linea en PlantaReciclaje para registrar el nuevo contenedor. Las clases base (IResiduo, Residuo, Contenedor) permanecieron intactas, validando que el diseno es extensible sin romper lo existente.

2. Patrones de diseno

2.1 Factory Method (ResiduoFactory)

La clase **ResiduoFactory** encapsula toda la logica de creacion de objetos Residuo. Expone dos metodos estaticos: **crearAleatorio()**, que selecciona un tipo al azar y genera atributos realistas (peso entre 0.1 y 10 kg, 10%% de probabilidad de toxicidad), y **crearPorTipo(TipoResiduo)**, que recibe un enum y devuelve la instancia concreta correspondiente.

Gracias a este patron, ni la Vista ni el Controlador necesitan conocer las clases concretas (ResiduoPlastico, ResiduoVidrio, etc.). El boton 'Simular Entrada de Residuo' simplemente invoca modelo.simularEntrada(), que internamente delega en la factoria. Para anadir un nuevo tipo, basta con crear la subclase y anadir un case en la factoria, sin tocar el controlador ni la vista.

2.2 Modelo-Vista-Controlador (MVC)

El proyecto esta organizado en tres paquetes estrictos:

Capa	Paquete	Responsabilidad
Modelo	modelo/	Logica de negocio pura: residuos, contenedores, factoria, persistencia. No importa javax.swing.*.
Vista	vista/	GUI Swing: pinta componentes, captura eventos y los expone via metodos addXxxListener(). No calcula l
Controlador	controlador/	Media exclusivamente: recibe eventos de la Vista, invoca operaciones en el Modelo y ordena a la Vista ac

El flujo tipico es: (1) el usuario pulsa un boton en la Vista; (2) el listener del Controlador se dispara; (3) el Controlador llama al Modelo; (4) el Controlador lee el resultado del Modelo y actualiza la Vista. En ningun momento la Vista accede directamente al Modelo ni el Modelo conoce la Vista.

3. Manual de usuario

3.1 Compilacion y ejecucion

```
javac -d out src/main/java/modelo/*.java src/main/java/vista/*.java  
src/main/java/controlador/*.java src/main/java/Main.java  
java -cp out Main
```

3.2 Interfaz grafica

La ventana principal se divide en cuatro areas:

Cinta Transportadora (panel izquierdo): muestra la lista de residuos pendientes de procesar, con su ID, tipo, peso y estado de reciclabilidad. Se actualiza cada vez que se simula una entrada o se procesa un residuo.

Contenedores (panel derecho): cuatro barras de progreso horizontales (Plastico, Vidrio, Papel, Metal) que muestran el porcentaje de llenado en tiempo real. Cuando un contenedor supera el 90%%, la barra se tinte de rojo. Cada contenedor tiene un boton 'Vaciar' para resetearlo a 0 kg.

Registro de Operaciones (panel inferior): log cronologico de todas las acciones realizadas en la sesion.

Botones de accion: 'Simular Entrada de Residuo' genera un residuo aleatorio via la factoria y lo anade a la cinta. 'Procesar Residuo' toma el primer residuo de la cinta, lo clasifica y lo deposita en su contenedor correspondiente (si es reciclable y cabe).

3.3 Persistencia

Al cerrar la ventana, el sistema guarda automaticamente los niveles de todos los contenedores en **estado_planta.json**. Al volver a abrir la aplicacion, se recuperan dichos niveles para retomar la sesion donde se dejo. Adicionalmente, cada residuo depositado se registra en **recycle.log** con fecha/hora, ID, tipo, peso y contenedor destino.